

CPE333 Operating Systems (1/2026)

Problem Session 3: Process Manipulation and Monitoring

- | | |
|-----------------------------|-----------------|
| 1. Name: Thanaboon Tikaew | ID: 67070501021 |
| 2. Name: Chanon Lhumsa-ard | ID: 67070501059 |
| 3. Name: Theeraphat Jaingam | ID: 67070501063 |
| 4. Name: Siriwan Yindeephon | ID: 67070501075 |

Environment: Ubuntu 24.04 on WSL2, gcc 15.2.0, 8 CPUs

1 Task 1: Important Linux Commands for Process Manipulation

1.1 Difference between top and ps

Both `top` and `ps` are used to inspect active processes on a Linux system, but they serve different operational purposes.

ps	top
Static snapshot: Prints a point-in-time snapshot of processes and exits immediately.	Dynamic monitor: Continuously refreshes and displays real-time system activity.
Batch / Scripting friendly: Ideal for scripting, pipelines, and grep (e.g., <code>ps aux grep ssh</code>).	Interactive: Stays active until user exits (typically by pressing <code>q</code>).
Complete listing: Outputs all matching processes in standard output stream.	Dashboard view: Summarizes top consumers fitting the current terminal dimensions.
Non-sorting by default: Requires specific CLI flags to sort by resource metrics.	Sorted automatically: Sorts by CPU or memory usage by default.
Zero persistent overhead: Exits immediately after reading <code>/proc</code> .	Continuous overhead: Consumes minor CPU while periodically refreshing.

Table 1: Comparison between `ps` and `top`.

Usage Example:

```
$ ps aux          # one-time process snapshot
$ top             # live, interactive monitoring dashboard (quit
                  with 'q')
```

1.2 Purpose of the nice command

The `nice` command launches a program with a modified *scheduling priority*. Each process is assigned a “niceness” value determining how the kernel scheduler prioritizes CPU time allocations. A higher niceness represents a lower priority (the process is “nicer” to other tasks), whereas a lower (or negative) niceness elevates execution priority.

- **Niceness range:** From -20 (highest priority) to $+19$ (lowest priority).
- **Default niceness:** 0.
- **Privileges:** Only root/privileged users can assign negative niceness values.

```
$ nice -n 10 ./my_program      # run with niceness 10 (lower priority)
$ sudo nice -n -5 ./my_program # run with niceness -5 (higher priority, root
                             # required)
$ renice -n 5 -p 1234          # adjust priority of running process PID 1234
```

1.3 How to kill processes and jobs

The `kill` command delivers signals to processes identified by their Process ID (PID) or Job ID. The most frequently used signals include `SIGTERM` (15) for graceful termination and `SIGKILL` (9) for immediate unconditional termination.

```
$ ps aux | grep my_program      # identify target process PID
$ kill 1234                     # send SIGTERM (graceful shutdown)
$ kill -9 1234                  # send SIGKILL (forceful immediate termination)
$ killall my_program            # terminate all processes with matching name
$ pkill my_program              # terminate processes matching regex pattern
$ jobs                          # view current shell background/stopped jobs
$ kill %1                       # terminate job 1 from job table
```

```
$ ps aux | grep ss1_1
ssiriwan 5678 0.0 0.1 12345 2345 pts/0 S 10:00 0:00 ./ss1_1.sh
$ kill 5678                     # gracefully terminate script process
$ kill -9 5678                  # forcibly terminate process if SIGTERM is
    blocked
```

2 Task 2: Running Shell Scripts in Foreground and Background

2.1 First situation: ss1_1.sh

The script `ss1_1.sh` performs a 10-second sleep loop:

```
#!/bin/bash
for i in $(seq 1 10); do
    sleep 1
done
```

Running in Foreground (`./ss1_1.sh`)

- The process occupies the terminal standard input/output streams.
- The shell prompt is blocked until the script completes (10 seconds).
- The process can be interrupted directly via `Ctrl+C` (`SIGINT`).

```
$ ./ss1_1.sh          # prompt blocked for ~10s, then returns
```

Running in Background (`./ss1_1.sh &`)

- The trailing `&` tells the shell to fork the process without blocking the parent shell.
- The shell outputs the job ID and PID (e.g., `[1] 5678`) and prompt returns immediately.
- The user can execute further commands concurrently while the script runs in the background.

```
$ ./ss1_1.sh &
[1] 5678          # job number [1], PID 5678, prompt returns immediately
```

Discussion

- **Foreground execution:** The shell performs a `wait()` system call for the child process, preventing interactive user input until termination.
- **Background execution:** The shell creates the child process but does not block on `wait()`, returning control to the terminal input loop immediately.

2.2 Second situation: ss1_2.sh

The script `ss1_2.sh` sleeps for 1000 seconds:

```
#!/bin/bash
sleep 1000
```

Suspending Process with `Ctrl+Z`

Pressing `Ctrl+Z` delivers the `SIGTSTP` signal to the foreground process group. The process is paused (state `T` / `Stopped`) and added to the shell's job table without being killed.

```
$ ./ss1_2.sh
^Z
[1]+  Stopped                  ./ss1_2.sh
$ jobs
[1]+  Stopped                  ./ss1_2.sh
```

Resuming with fg and bg

- **fg %1**: Resumes the suspended job in the **foreground** by sending SIGCONT. The terminal prompt blocks until completion.
- **bg %1**: Resumes the suspended job in the **background** by sending SIGCONT. The prompt remains available for further commands.

```
$ fg %1      # resume job 1 in foreground
$ bg %1      # resume job 1 in background
```

3 Task 5: STCF (Shortest Time to Completion First) Scheduler

3.1 Algorithm Overview

Shortest Time to Completion First (STCF), also referred to as *Shortest Remaining Time First (SRTF)*, is a **preemptive** CPU scheduling policy. At every discrete time unit, the scheduler evaluates all available processes that have arrived and allocates the CPU to the process with the shortest remaining burst time.

- **Zero-overhead model:** Scheduler evaluation and context switching take zero CPU time.
- **Arrival boundary:** A process arriving at time T is available at time $T - 1$'s completion, ready for evaluation at time T .
- **Tie-breaking rule 1:** If multiple processes arrive simultaneously, prioritize by lowest `process_id`.
- **Tie-breaking rule 2:** If multiple active processes share identical remaining burst time, prioritize by lowest `process_id`.

3.2 Implementation

```

1  /*
2  * Scheduler function for PS3.c (STCF / Shortest Remaining Time First)
3  * processes[] : array of processes (pid, arrival, burst)
4  * n           : number of processes
5  */
6  void stcf(struct Process p[], int n) {
7      int remaining[MAX];
8      int completed = 0;
9      int t = 0;
10
11     /* Initialize remaining burst times */
12     for (int i = 0; i < n; i++)
13         remaining[i] = p[i].burst;
14
15     while (completed < n) {
16         int chosen = -1;
17
18         /* Select available process with minimum remaining burst time;
19          resolve ties using lowest pid */
20         for (int i = 0; i < n; i++) {
21             if (p[i].arrival <= t && remaining[i] > 0) {
22                 if (chosen == -1 ||
23                     remaining[i] < remaining[chosen] ||
24                     (remaining[i] == remaining[chosen] && p[i].pid < p[chosen].
25                     pid)) {
26                         chosen = i;
27                 }
28             }
29         }
30
31         if (chosen == -1) {
32             t++; /* CPU idle: advance time */
33             continue;
34         }
35
36         /* Execute chosen process for 1 time slice */
37         printf("Time %2d-%2d : Process %d\n", t, t + 1, p[chosen].pid);
38         remaining[chosen]--;
39         if (remaining[chosen] == 0)

```

```
39         completed++;  
40         t++;  
41     }  
42 }
```

3.3 Test Case Results

3.3.1 Case 1 (case1.csv)

Input Specification		
PID	Arrival	Burst
1	0	7
2	2	4
3	4	1

Output Timeline	
Time Slot	Running Process
0–1	Process 1
1–2	Process 1
2–3	Process 2
3–4	Process 2
4–5	Process 3
5–6	Process 2
6–7	Process 2
7–8	Process 1
8–9	Process 1
9–10	Process 1
10–11	Process 1
11–12	Process 1

3.3.2 Case 2 (case2.csv)

Input Specification		
PID	Arrival	Burst
1	0	2
2	5	3
3	5	3
4	6	1

Output Timeline (CPU Idle: 2–5)

Time Slot	Running Process
0–1	Process 1
1–2	Process 1
2–5	(CPU Idle)
5–6	Process 2
6–7	Process 4
7–8	Process 2
8–9	Process 2
9–10	Process 3
10–11	Process 3
11–12	Process 3

3.3.3 Case 3 (case3.csv)

Input Specification		
PID	Arrival	Burst
2	0	5
1	3	2
3	6	1

Output Timeline	
Time Slot	Running Process
0–1	Process 2
1–2	Process 2
2–3	Process 2
3–4	Process 1
4–5	Process 1
5–6	Process 2
6–7	Process 2
7–8	Process 3

3.4 Discussion

- **Case 1 (Preemption Flow and Short-Job Prioritization):** Process 1 executes from $t = 0-2$ (remaining burst: 5), whereupon Process 2 arrives with burst 4 and immediately preempts Process 1 ($4 < 5$), followed by Process 3 arriving at $t = 4$ (burst 1) and preempting Process 2 ($1 < 2$), which demonstrates how STCF enforces dynamic multi-level preemption to minimize response and waiting times for newly arriving short tasks before allowing preempted processes (Process 2, then Process 1) to resume and finish at $t = 7$ and $t = 12$ respectively.
- **Case 2 (CPU Idle State and Simultaneous Arrival Resolution):** After Process 1 completes at $t = 2$, the CPU remains idle from $t = 2-5$ until Process 2 and Process 3 arrive simultaneously with identical bursts of 3, where Process 2 is selected first via the lower-PID tie-breaking rule ($PID\ 2 < PID\ 3$) and runs until $t = 6$ before being preempted by Process 4 (burst $1 < 2$), demonstrating that the scheduler robustly handles non-contiguous arrival gaps while applying deterministic tie-breaking to schedule simultaneous jobs before completing Process 2 ($t = 7-9$) and Process 3 ($t = 9-12$).
- **Case 3 (Tie-Breaking Behavior on Equal Remaining Time):** Process 2 executes from $t = 0-3$ (remaining: 2) until Process 1 arrives with burst 2, triggering a preemption because Process 1 has the lower PID ($PID\ 1 < PID\ 2$), yet when Process 3 arrives at $t = 6$ (burst 1) while the resumed Process 2 also has 1 remaining unit, Process 2 is allowed to continue running without preemption ($PID\ 2 < PID\ 3$), which proves that the PID tie-breaking rule functions symmetrically to both preempt a running process and protect an ongoing execution whenever candidate processes share identical remaining execution times.
- **Summary:** Overall, the STCF scheduler guarantees optimal average turnaround time by continuously dispatching the job closest to completion, seamlessly idling during arrival gaps, and resolving all collision scenarios through consistent PID-based tie-breaking.